

News Feed

Things Get a little complicated?

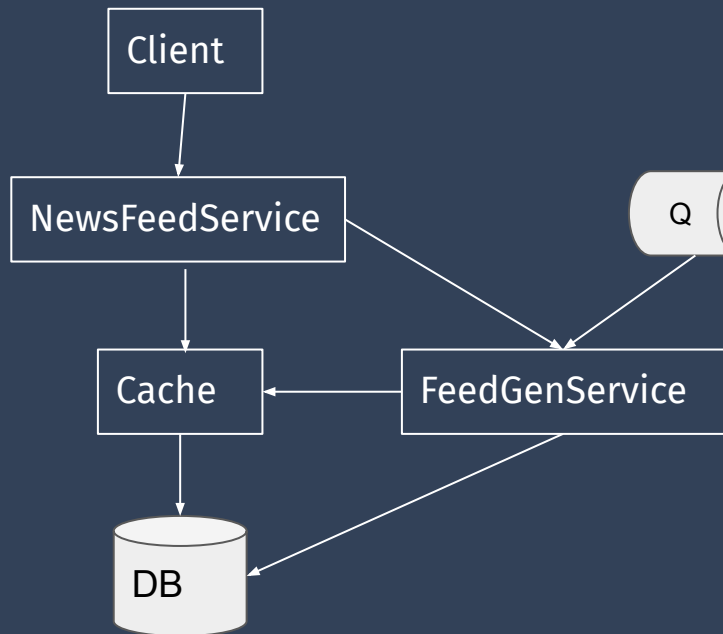
How is this problem different from Tiny URL?



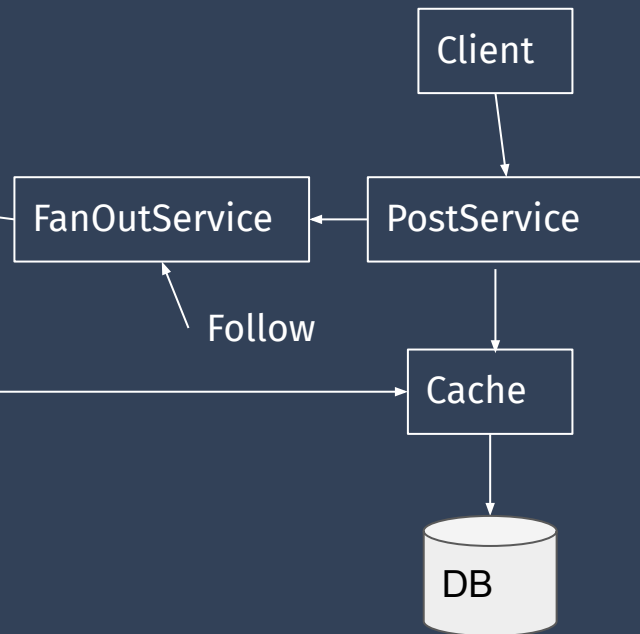
INTERVIEW
KICKSTART

Logical Architecture (pretending we have 1 server)

News Feed Generation



PostService



Step 4: NewsFeedService

1. System should generate a news feed for the user, based on who the user follows.
 - a. NewsFeedService should fetch top X (say 100) latest, most popular and relevant photos of the people that the user follows.
 - i. See if the info is available in cache. If not:
 - ii. Get a list of people the user follows.
 - iii. Fetch metadata of each user's latest 100 photos.
 - iv. Rank the photos (recency, what user liked, location etc).
 - v. Return top 100 of that subset.
 - vi. Store the list in cache
 - b. Alternatively, FeedGenService could pre-generate the same and store it in UserNewsFeed table and in cache.
 - i. Generate new data based on last feed generation timestamp in the table.



Step 4: NewsFeedService

1. How should the user fetch the news feed?
 - a. Pull model: Clients pull the news feed from servers manually, or at a regular interval.
 - i. Pro: Less load on system while following celebrities.
 - ii. Con: Empty pull requests if no data.
 - b. Push model (long poll or websockets): Servers push new data to users when it becomes available.
 - i. Pro: No wasted cycles from client in empty responses.
 - ii. Con: Too much data to push for updates from celebrities.
 - c. Hybrid: Pull model for pulling data from celebrities and push model for rest of us mere mortals.

Step 4: NewsFeedService

Cache Tier

How to organize the news feed in cache?



INTERVIEW
KICKSTART

Step 4: NewsFeedService

DB Tier

UserNewsFeed table: Primary Key = FeedItemId(int64).

Other columns = UserId(int64), PhotoId(int64), PhotoOwnerId(int64), Description(varchar 64),
ImgStoreOrCDNPath(varchar 256), FeedCreationTimestamp(datetime), NumLikes(int32),
NumComments(int32))

Let's assume we store 3 days worth of news feed items per user, 1000 feeds per day.

How do you cap it at 3 days?



Step 5: Need for Scale in NewsFeedService:

- For each microservice
 - A deterministic set of reasons can be posed across all interviews
 - Need to scale for storage (storage and cache tiers) - Yes
 - Need to scale for throughput (CPU/IO) - Yes
 - Need to scale for API parallelization - No
 - Need to remove hotspots - No
 - Availability and Geo-distribution - Yes
 - Solve algebraically first and then put numbers
 - Total users $\approx$ 1 billion
 - Daily active users $\approx$ 500 million
 - Photo views per second $\approx$ 20000
 - Avg num of timeline fetches a day by each active user $\approx$ 10
 - Average photo file size $\approx$ 200kb
 - Let's look at each tier separately.

Step 5: Need for Scale in NewsFeedService:

- App Tier
 - Timeline fetch req = 500 million daily active users * 10 fetches a day = 5 billion a day
 - $= 5,000,000,000 / 100,000$ (approx sec in a day) = 50,000
 - Using previous formula, roughly **30000/t API calls per server.**
 - Since we are not sending the photo, but sending a batch of meta-data, let's assume that each fetch takes around 100 ms.
 - Plug in $t = 100$ ms in above formula = $30000/100 = 300$ rps per server
 - For 50000 photos per sec ≈ 167 servers.

Step 5: Need for Scale in NewsFeedService:

- Cache Tier
 - Let's assume we store 500 news feed items per active user.
 - So $500 \text{ million users} * 500 \text{ items} * 300 \text{ bytes per item} = 75,000,000,000,000 \text{ bytes} \approx 75 \text{ TB}$
 - For easier calculations, assume 75 GB RAM per server
 - So $75 \text{ TB} / 75 \text{ GB} \approx 1000 \text{ servers}$
 - How do you store it in these server?

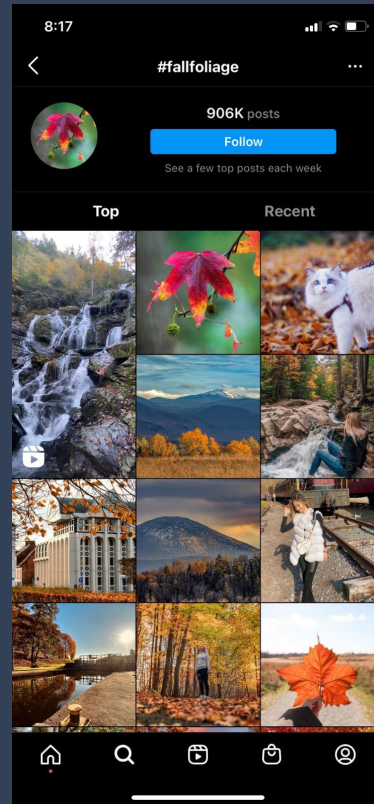
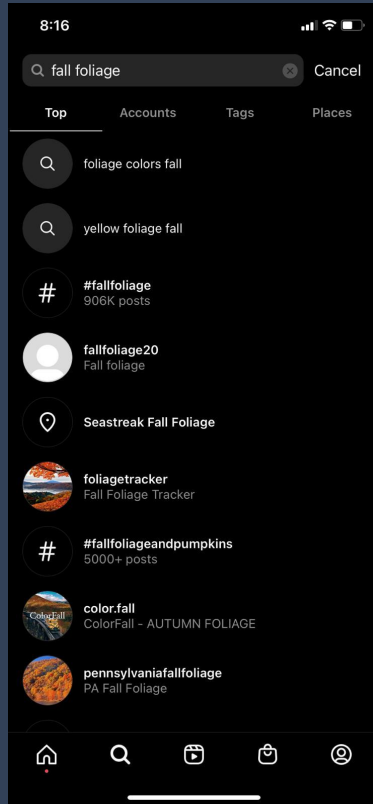
Step 5: Need for Scale in NewsFeedService:

- DB Tier
 - Let's assume we store 3 days worth of news feed items per user, 1000 feeds per day.
 - So $1 \text{ billion users} * 3000 \text{ items} * 300 \text{ bytes per item} = 900,000,000,000,000 \text{ bytes} \approx 900 \text{ TB}$
 - Assume 4.5 TB per DB shard
 - So $900 \text{ TB} / 4.5 \text{ TB} \approx 200 \text{ DB servers}$

Step 6: Distributed Architecture for NewsFeedService

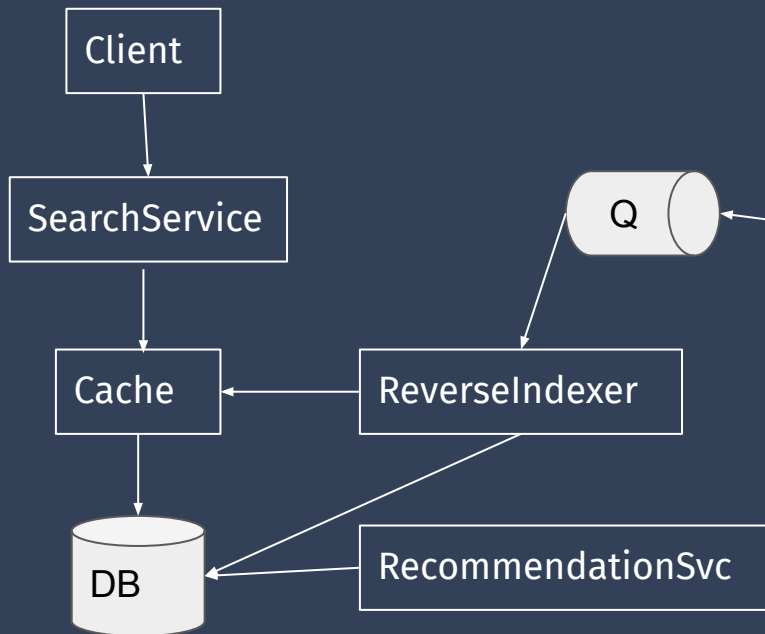
- For App tier, round robin would suffice unless you are holding the cache on the same servers
- For cache tier, shard based on User ID
- For DB tier as well, shard based on User ID
- Explain how APIs will work in this sharded setting

Search Service

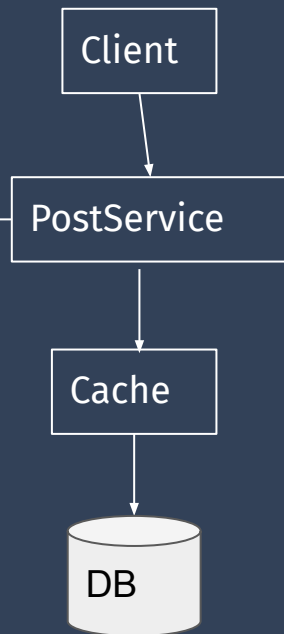


Logical Architecture (pretending we have 1 server)

Search Service



PostService



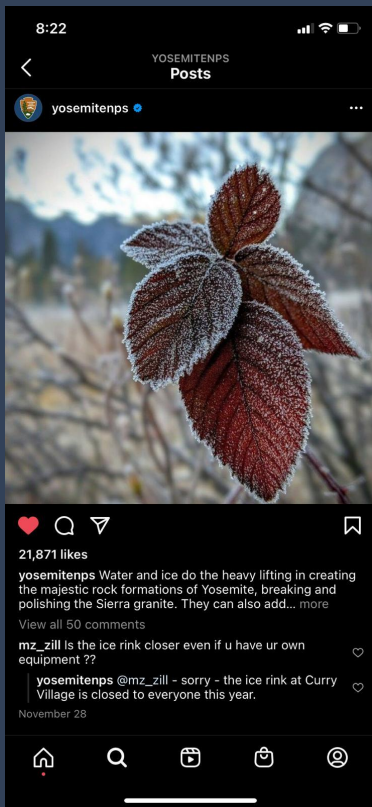
Trends, User
behavior etc.

Step 4: Search Service

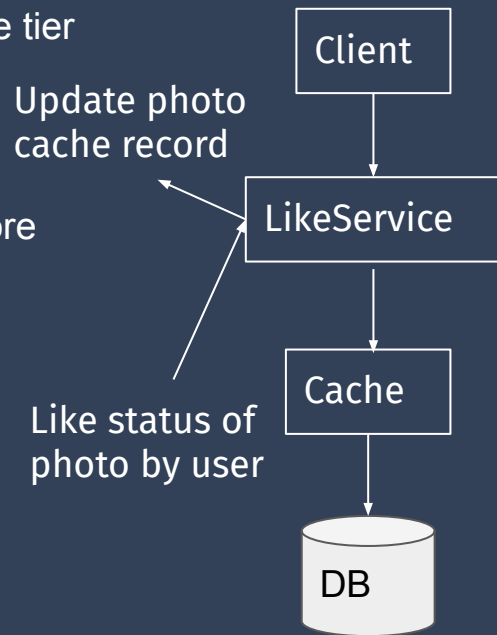
1. Search Service

- a. Search based on accounts.
 - i. Can search in UserTable, using index on name.
 - ii. Maintain results in cache to improve performance.
- b. Search on keywords
- c. Search on hashtags
 - i. For both keywords and hashtags, an offline process will have to generate a reverse index of keyword/hashtag -> sorted list of photo ids.
 - ii. Everytime a new photo is posted, this list needs to be updated, but doesn't need to get reflected immediately.
 - iii. You could also mention recommendation service, which will generate recommendations based on user's previous likes, current trends etc. This is what you see when you first click on search button, w/o entering search term.

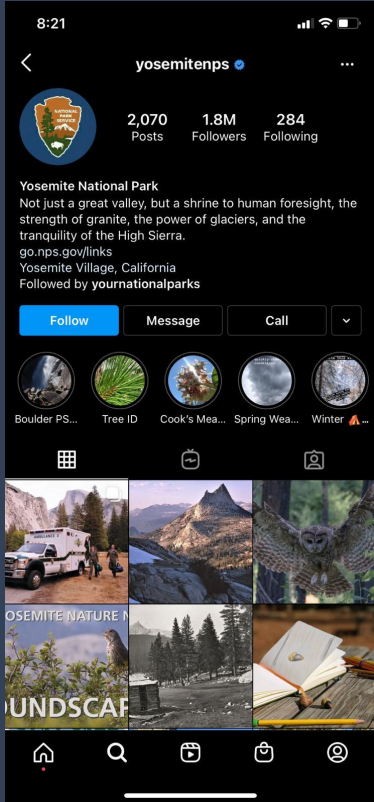
Step 4: LikeService



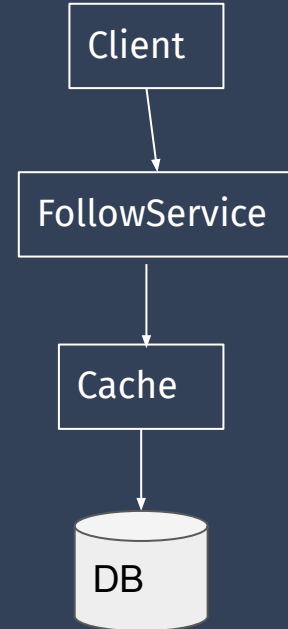
- Tiers: App server tier, Cache or In-memory tier, Storage tier
- Data Model: K-V pair
 - K: UserID+PhotoID: V: time_stamp
- How to store in Cache tier - Hashmap
- How to store in Storage tier - as a row oriented K-V store
- APIs
 - likePhoto(userID, photoID, time_stamp, ...)
 - getLikeStatus(userID, photoID)
- Algorithm in APIs
 - Create(K.V) - CRUD
- Flow of APIs
 - Create a record in the likes table
 - Update aggregate likes count of the photo
 - Offline process or sharded counter?
 - Async fanout to update recommendation logic



Step 4: FollowService



- Tiers: App server tier, Cache or In-memory tier, Storage tier
- Data Model: K-V pair
 - K: UserID+UserID: V: time_stamp
- How to store in Cache tier - Hashmap (userid-># of followers)
- How to store in Storage tier - as a row oriented K-V store
- APIs
 - follow(userID, userID), unfollow(userID, userID)
 - doesUserFollow(userID, userID)
- Algorithm in APIs
 - Create(K,V) - CRUD
 - update/delete(K,V) - CRUD
- Flow of APIs
 - Check if follow count is within limits
 - Create a record in follow table
 - Update cache with num of followers
 - Async message to update new feed generation logic
 - Similar flow for unfollow



Problem 2

Online System - Design an Uber clone



INTERVIEW
KICKSTART

Uber Screenshots



REQUEST

Tap to select location



RIDE

Input or tell your driver
your destination



RATE

Provide immediate feedback
at the end of each ride

Step 1: Functional Requirements

- Rider can request for a trip, should be able to give options
 - Book in advance, or real time
- Rider should get cost and an ETA
- User Account Management for Riders and Drivers
- Location based Multiple vehicle Dashboard for Rider
- Driver should be able to accept or reject a trip request
 - Notifications to the rider and the driver
- Routing
- Once trip is set, rider should be able to check the progress of the vehicle
- Rider pays driver
- Rider and driver endorse each other

Step 1: Non-Functional requirements

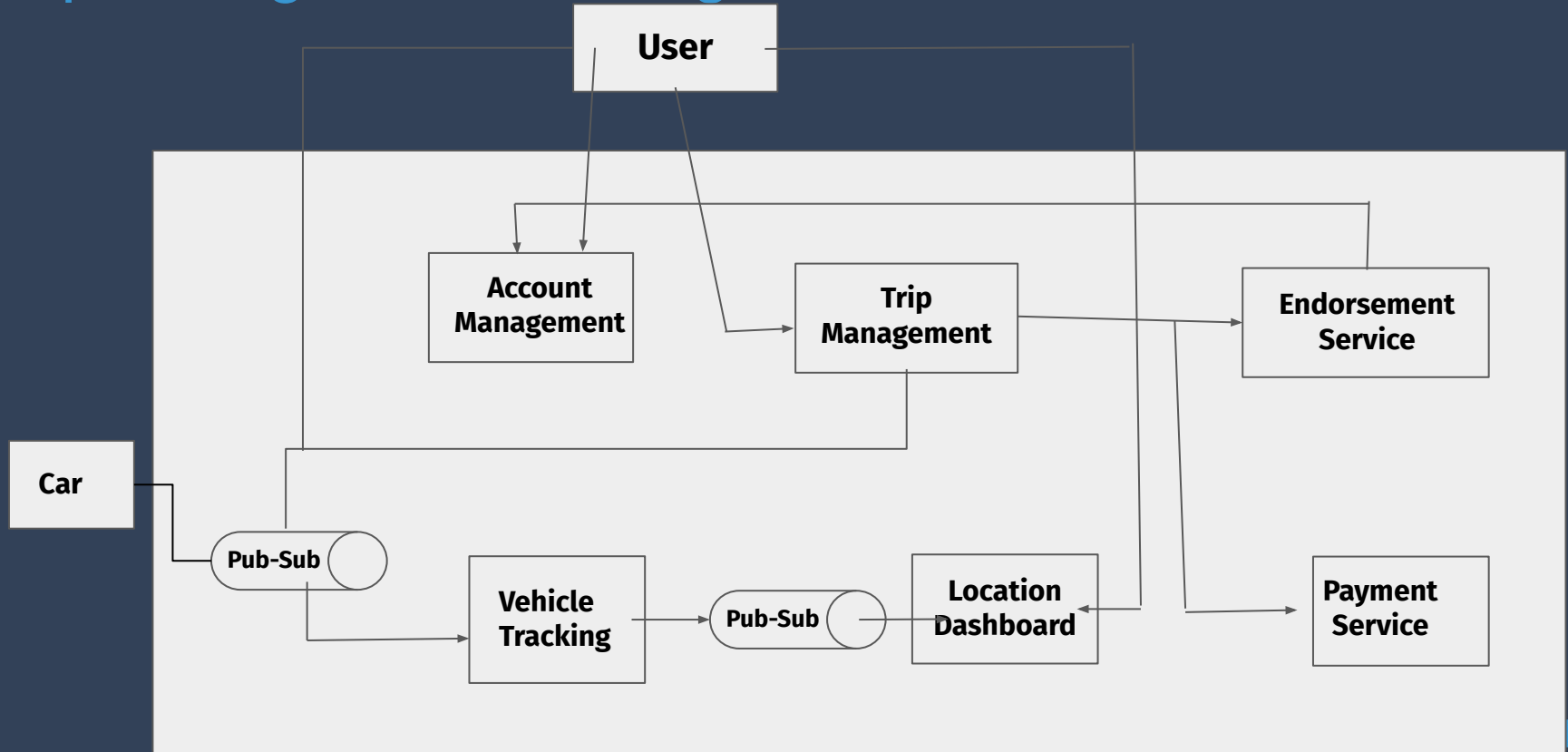
- Number of riders and drivers: 80 million riders + 4 million drivers
- Number of vehicles registered ~ 4-5 million
- Number of trips per month: 40 million

Step 2: Defining Microservices

- Account Management Microservice (Riders + Drivers)
- Vehicle Tracking Microservice
- Location based Dashboard Microservice
- Trip Management Microservice - ETA
- Payment Management Microservice
- Endorsement Management Microservice

Clearly, a breadth oriented problem

Step 3: Logical block diagram



Microservice to focus for Uber

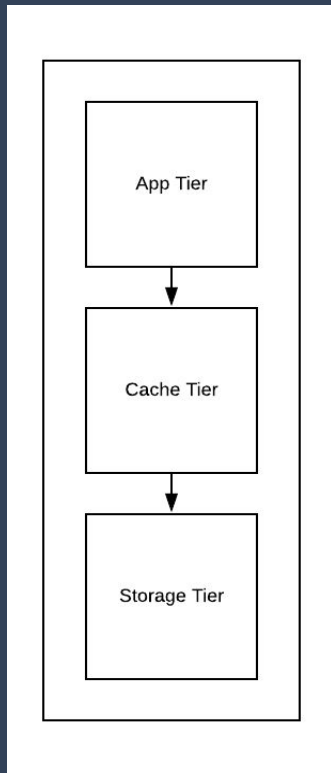
- Vehicle Tracking Microservice
- Location based Dashboard Microservice
- Trip Management Microservice

GeoHash:- How are number of available cabs found

- The earth is a sphere. It's hard to do summarization and approximation based purely on longitude and latitude.
- So Uber divides the earth into tiny cells using the Google S2 library. Each cell has a unique cell ID.
- When Uber needs to find the cabs near a location, a circle's worth of coverage is calculated centered on where the rider is located.
- Uber builds the map overtime by:
 - Identifying the missing roads and incorrect data.
 - Preferred access points at large locations such as airports, universities etc.

Step 4: Vehicle Tracking Microservice

- **Tiers:** App server tier, Cache or In-memory tier, Storage tier
- **Data Model:** K-V pair
 - K: VIN: V: *location*, ts, status, geo-hash, prev geo-hash, other properties
 - Loc_table: geohash, top_left_x, top_left_y, bot_right_x, bot_right_y
- **How to store in Cache tier** - Hashmap
- **How to store in Storage tier** - as a row oriented K-V store
- **APIs**
 - create(vin, lat, long) when registered
 - updateLoc (vin, lat, long)
- **Algorithm in APIs**
 - Simple K-V workload to update current location
 - Calculate geohash based on location (look up geo spatial index)
- **Flow of APIs**
 - App server tier consumes new vehicle registration from driver and location updates from pub-sub
 - Sends to cache server tier
 - Write back caching
 - Update location dashboard service if change in geo hash



Step 5: Vehicle Tracking Microservice

- Need to scale for storage ? Maybe, depending on how many days of history
 - Let's assume we have 2 different tables:
 - To persist current location for persistence if cache goes down
 - To save history for audit/compliance purpose - No need of second level granularity. We can assume we save 1 min snapshot for up to 1 year.
 - Amount of storage in cache and storage tiers: Number of vehicles * size of K-V pair per vehicle
 - For 1st table: 5 million vehicles * 40 bytes per vehicle (vin + ts + lat + long + 2 geo-hash etc) = ~200 mb
 - For 2nd table: 60 points an hr * 24 * 365 * 200 mb ~= 100 TB
- Need to scale for throughput ? yes
 - Number of update API calls per second: thousands per sec
 - Cannot be handled by any single server
- Need to scale for API parallelization ? no
 - APIs themselves are constant latency, so no need to parallelize
- Availability and Geo-location ? yes

Step 5: Vehicle Tracking Microservice

- App Tier
 - Let t = CPU work time in ms from a single thread
 - So number of operations per second handled by single thread = $1000/t$
 - Number of concurrent threads in a commodity server = 100-200 (determined by experiments)
 - So the number of API calls a server can handle per second $\sim 100000/t$, operating at full capacity.
 - But typical servers operate at 30-40% capacity, so roughly **30000/t API calls per server**.
 -
 - For a simple location update + calculating geo-hash, assume it will take around 20 ms
 - Plug in $t = 20$ ms in above formula = $30000/20 = 1500$ rps per server
 - 5 million cars updating location every 5 sec, assume 1 mil updates per sec.
 - For 1,000,000 updates per sec $\sim 1,000,000 / 1,500 \sim 667$ servers (distributed globally).

Step 5: Vehicle Tracking Microservice

- Cache Tier
 - In memory data per vehicle:
VIN (~8b) + lat (4b) + long (4b) + 2 geohash (20b) + status (1b) + other (3b)
~= 40 bytes per vehicle
 - Number of vehicles ~= 5 million
 - Total memory requirement = $5,000,000 * 40$ ~= 200,000,000 ~= 200 mb
 - Can fit into 1 server, but no point in storing NYC/SF/India car info in 1 server
 - Distribute the data across multiple data centers based on Geo location.

Step 5: Vehicle Tracking Microservice

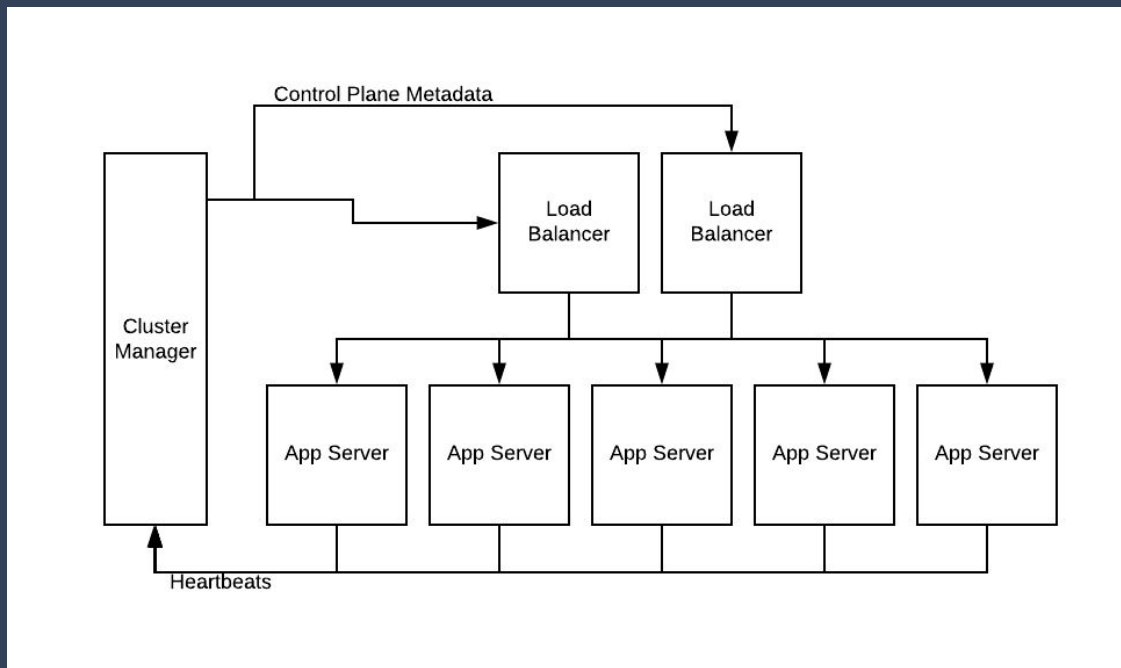
- DB Tier
 - In memory data per vehicle:
VIN (~8b) + ts(4b) + lat (4b) + long (4b) + status (1b) + other (3b)
≈ 25 bytes per vehicle (we don't need to store geo-hash in the db)
 - Number of vehicles ≈ 5 million
 - Total memory requirement = $5,000,000 * 25 \approx 125,000,000 \approx 125 \text{ mb}$ per data pt.
 - If you assume that you are persisting this info for compliance purpose, you don't need second level granularity
 - For minute level aggregation of 1 year data:
 $125 \text{ mb} * 60 * 24 * 365 \approx 125 \text{ mb} * 500,000 \approx 62,500,000 \text{ mb} \approx 63 \text{ TB}$
 - Assuming 4TB per db server, you will need ~16 shards (w/o replication)
 - Similar to cache, doesn't make sense to hold data for all vehicles worldwide in a single cluster, distribute it across multiple data centers.

Step 6: Vehicle Tracking Microservice

- Sharding
 - Horizontal sharding
 - Map subsets of vehicles to a single shards
 - Shard based on geo location
 - If you need further sharding, shard based on hash of VIN#
- Placement of shards in servers
 - Consistent hashing
- Replication
 - Yes for availability as well as for throughput
- CP or AP?
 - AP: does not need to be strictly nanosecond level consistent
 - Best configuration: $N = 3$ or more, $R = 2$, $W = 2$, high throughput, lowest latency

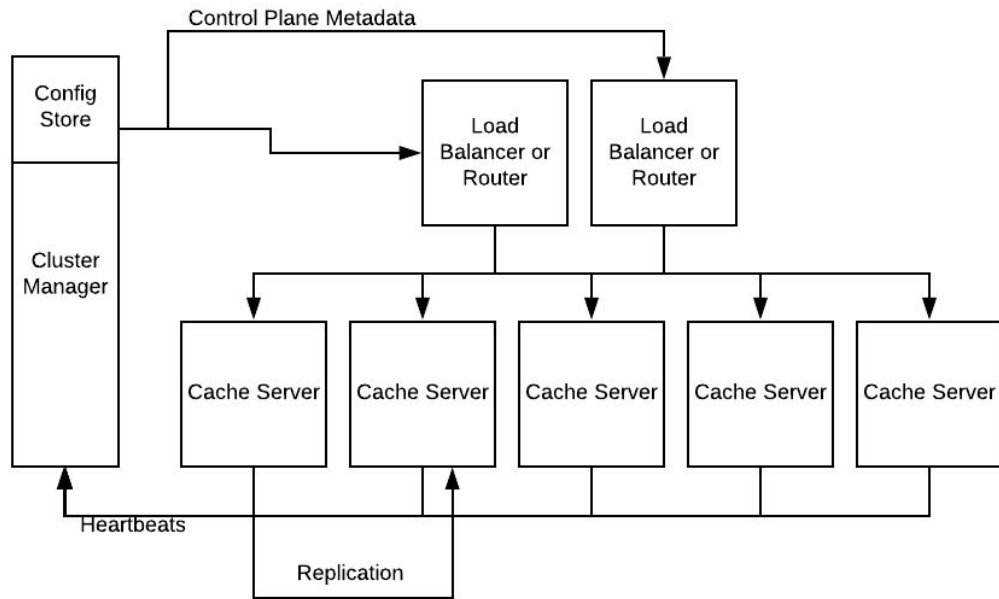
Step 6: Vehicle Tracking Microservice

Architectural layout for App tier



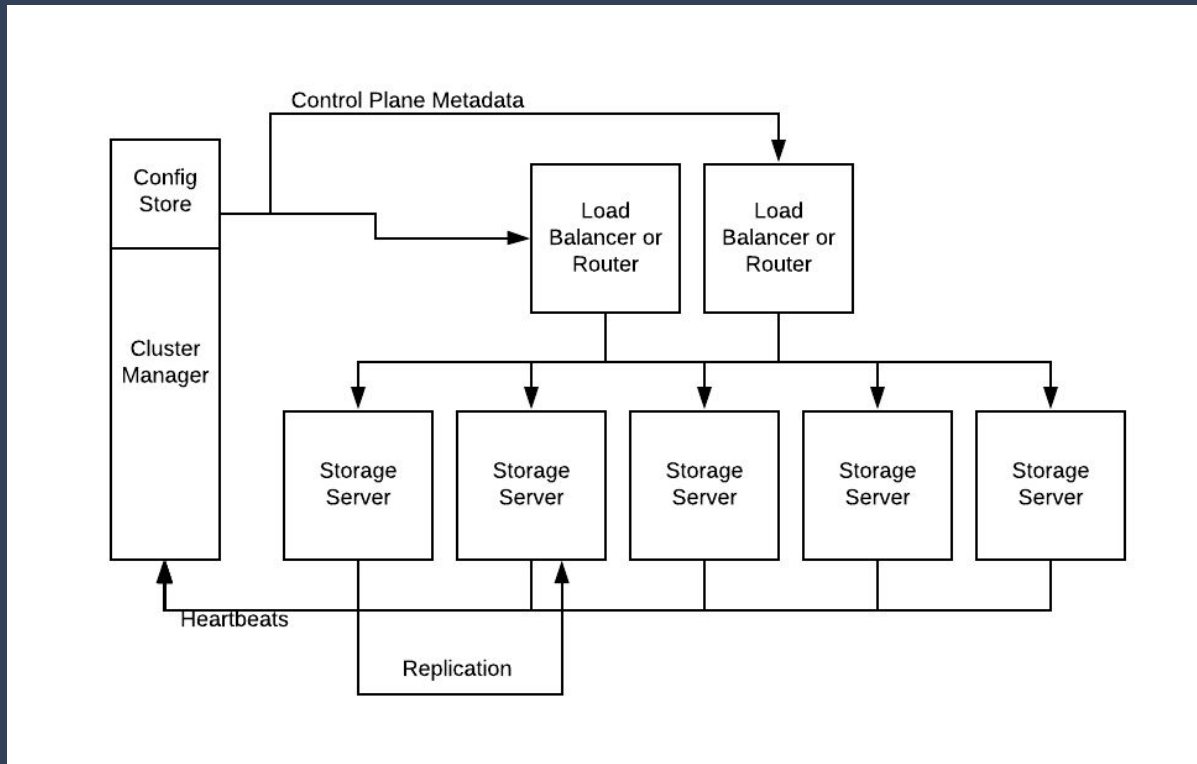
Step 6: Propose the Distributed Architecture

Architectural layout for cache tier



Step 6: Propose the Distributed Architecture

Architectural layout for storage tier



Step 4: Location based Dashboard Microservice

- **Tiers:** Cache tier, Storage tier (just for recovery)
- **Data Model:** K-V pair
 - K: Location Id: V: timestamp, *list of {vehicles + properties + location}*
 - Loc_adj_list: loc_id, north, south, east, west, ne, nw, se, sw
- **How to store in in-memory tier:**
 - Hash table: Geo hash as the key, list/map/ht of cars within that geohash as values
- **How to store in storage tier:**
 - Row oriented.
- **APIs:**
 - get(location id) -> all vehicles given location
 - get(location id, properties) -> search vehicles with certain properties given location
 - Subscribe(driver ids) -> to track the movement of cars in the vicinity

Step 5: Location based Dashboard Microservice

- Need to scale for storage ? yes
 - Amount of storage in cache and storage tiers: Number of unique location ids times size of each K-V pair
- Need to scale for throughput ? yes
 - Number of API calls per second: number of trips /sec * number of location views / trip + geo-hash update from cars.
 - Number of servers required per tier - calculations from steps discussed in a different deck
- Need to scale for API parallelization ? Maybe
 - APIs themselves are constant latency, so no need to parallelize
 - But internally, if you are requesting vehicles in the neighboring blocks, you can parallelize.
- Availability ? yes
- Geo-location based distribution ? yes

Step 5: Location based Dashboard Microservice

- Cache Tier

- What do we need to store in memory?
 - Key: Geohash (6 bytes) + timestamp (4 bytes) + values: List/Map/Hash Table of VIN (~8b) + lat (4b) + long (4b) + status (1b) + type (1b) = 18 bytes per vehicle (round up to 20)
 - Assume around 20 cars in 1 cell $\approx$ 400 bytes per cell
- Number of cells of 6 char geohash $\approx 32^6 \approx$ 1 billion
- Total memory requirement = $1,000,000,000 * 400 \approx 400,000,000,000 \approx$ 400 gb
- Assuming 1 server can store about 40 GB, you will need about 10 machines
- Other way to calculate:
- Area of US $\approx$ 10 million sq. km
- If you divide that into cells of 500m * 500m, you will get ~40 million cells

Step 6: Location based Dashboard Microservice

- Sharding
 - Horizontal + vertical sharding
 - Horizontal : Geo distributed (Shards in west coast containing geo hash near that region)
 - Vertical: Within a geo distribution, shard based on driver ids to distribute write ops:
 - Shard1 : GeoHash1 -> D1, D11
 - Shard2 : GeoHash1 -> D2, D12
- Placement of shards in servers
 - Consistent hashing
- Replication
 - Yes for availability as well as for API parallelization of fetching data from neighboring cells
- CP or AP?
 - AP: does not need to be strictly nanosecond level consistent
 - Best configuration: N = 3 or more, R = 2, W = 2, high throughput, lowest latency

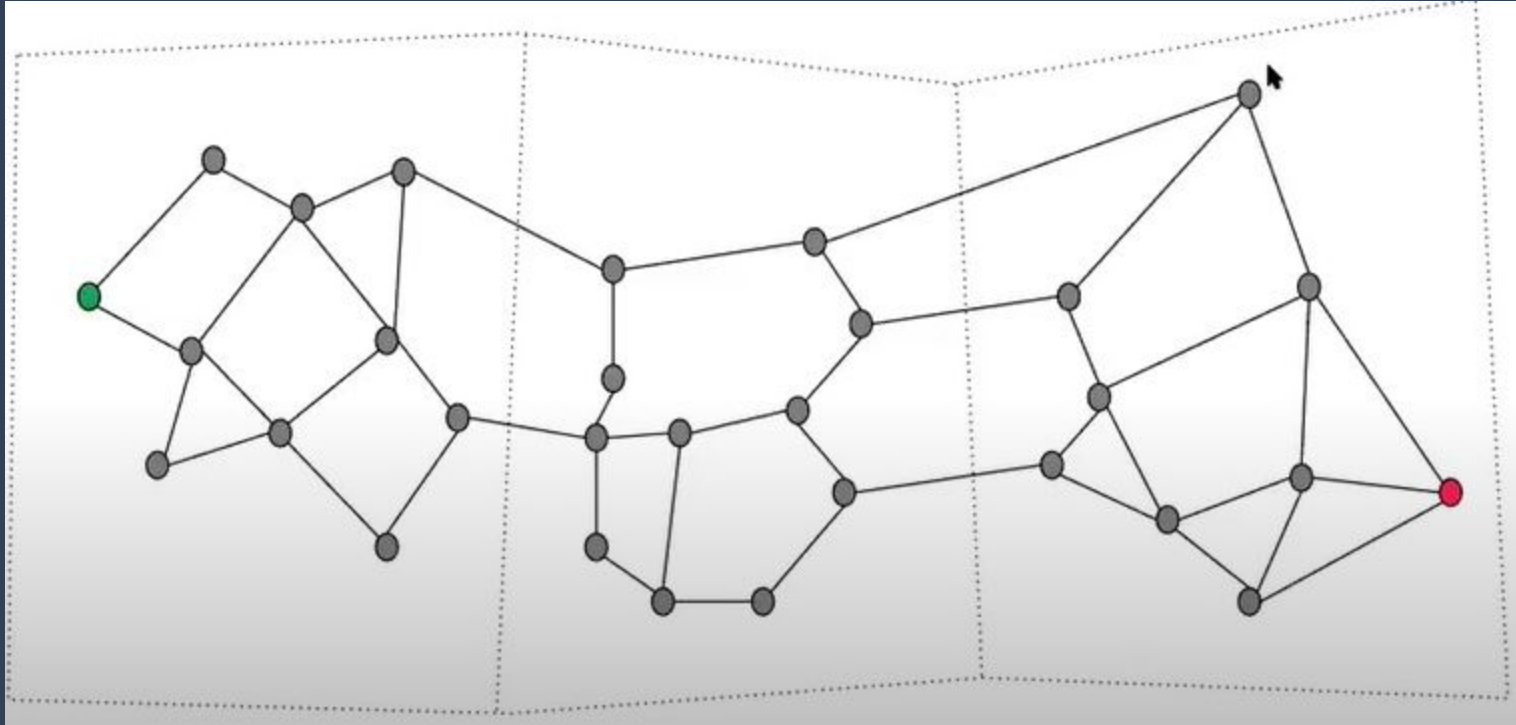
Step 4: Trip Management Microservice

- Could be divided into 2 parts
 - MapService to get map, calculate route, ETA
 - getMap, getRoute (let's assume they are handled by external APIs)
 - getTravelETA
 - TripInfo for operations around trip such as
 - requestTrip
 - acceptTrip
 - cancelTrip
 - rejectTrip
 - completeTrip
 - getTripStatus
 - tripHistory

How are ETA's calculated

- ETA is the estimated time of arrival of a cab.
- To calculate the ETA for a ride we need:
 - Track the available cabs nearby
 - Track the rides that are ready to finish the ride that might take less time
- Uber models the road network as a graph and has 4 layers of computations
 - Map data -> routing info (graph) -> traffic analysis -> ML
 - Gps data on road segments to calculate real time speed
 - Use traffic data to compute edge weights
 - ML to bridge gap between predicted ETA and actual ETA.

How are ETA's calculated



How are ETA's calculated

Identify

Identify pickup/dropoff coordinates for each location pin.



Compute

Compute geospatial proximity of location pin to pickup/dropoff coordinates.



Set

Set the **closest** pickup/dropoff to be preferred pickup-points.



Step 4: Trip Management Microservice

- TripInfo related operations:
- Data Model: K-V pair
 - K: <Trip id>: V: *rider, driver, startTime, endTime, price, properties, state of trip*
 - K: <driver/Trip id>: V: *rider, driver, properties*
 - K: <rider/Trip id>: V: *rider, driver, properties*
- How to store in Storage tier
 - Row oriented K-V store
- APIs
 - create(K, V) for model 1 when trip requested
 - update (K, V) for model 1 when when trip changes state
 - Update (K, V) for models 2 and 3 when trip completed
 - subscribe(driver_id) for location updates.
- Flow of APIs
 - Request trip -> app server calls Location service to get list of vehicles
 - Logic to find the best match
 - Optimize for overall wait time
 - Create trip
 - For each vehicle, send notification to driver
 - If driver accepts, update trip
 - Get driver's ETA

Step 5: Trip Management Microservice

- Need to scale for storage ? no
 - Amount of storage in cache and storage tiers: Number of trips per second * size of K-V pair per trip * sizing for a few years
 - 4,800,000,000 trips in 10 years * 40 bytes per trip $\approx$ 192 GB
- Need to scale for throughput ? no
 - Number of update API calls per second: few hundreds per sec
- Need to scale for API parallelization ? no
 - APIs themselves are constant latency, so no need to parallelize
- Availability ? yes
- Geo-location based distribution ? yes

Step 6: Trip Management Microservice

Architectural layout for every tier same as previous microservices

Servers needed only for replication



Price Surge in Uber - Optional

- The prices are increased when there is more demand and less supply.
- Surge is used to help to meet supply and demand as by increasing the price more cabs will be on the road when the demand is more.
- Uber uses “surge pricing algorithm” to determine the prices of rides based on the laws of supply and demand.
- This algorithm is ran every 5–10 minutes to determine the prices of rides for places of high demand and the price is determined by the availability of cars and the demand for the cars.
- The goal of the algorithm is to maximize the number of rides that can be provided (especially of times with low supply and high demand).

Cheat Sheet

- Redundancy - Replication and/or Snapshot.
- High write load/storage - sharding
- High read load - replication
- Scaling - Primary-Secondary or Leaderless
- High read/write concurrency - Duplicate record.
- Big objects - split into smaller chunks.
- Real time communication - Long polling / Websockets
- Search - index/primary key + reverse index
- Decoupling - message queues / pub-sub

External references

GeoHash Explorer: <http://geohash.gofreerange.com/>

Microservices blog: <https://microservices.io/>

High scalability blog: <http://highscalability.com/start-here/>

Numbers to know: <https://gist.github.com/jboner/2841832>

<http://highscalability.com/blog/2013/1/15/more-numbers-every-awesome-programmer-must-know.html>

Check out tech blogs by various companies

Books:

Building Microservices, Monolith to Microservices by Sam Newman

Patterns of Enterprise Application Architecture by Martin Fowler

Microservices Patterns by Chris Richardson

Any many more..

Graph image copied from: <https://www.youtube.com/watch?v=FEebOd-Pdwg>



**INTERVIEW
KICKSTART**

Thank you!

Credits: Omkar Deshpande and Niloy Mukherjee for their help and for allowing to use a few of their slides, Vivek for cheat sheet inspiration.



INTERVIEW
KICKSTART